The AI Commons Book

The guide to AI Commons: **1 IdP, 14 services, 4 compute blocks, 20 tools.** Start with the course *Welcome to Commons*; come here for the detail. Production is the default environment. Select dev only when the task explicitly requires it; every write still requires review and approval.

#	Chapter	Read it to
1	Platform overview	Get the mental model, URLs and tenants
2	Security and logging in	Log in, get tokens, understand IAM and secrets
3	The services	Know what each API does and its endpoints
4	The tools	Use the 20 tools and the path model
5	How it is wired together	Trace requests, runs, AI calls, config changes
6	Triggers	Run blocks manually, on schedule, by webhook
7	Setting up MCP	Connect Cursor and Claude to Commons
8	Skills and playbooks	Teach agents; install IDE skills
9	Building a datalake	Bronze → silver → gold, then query
10	Building a workflow	Task DAGs, bindings, runs
11	Get started examples	Nine hands-on production examples
12	Reference	URLs, topics, codes, glossary, further reading

Skills for Claude Code, Cursor and Codex ship in the `grasp-labs/commons` repository (chapter 8).

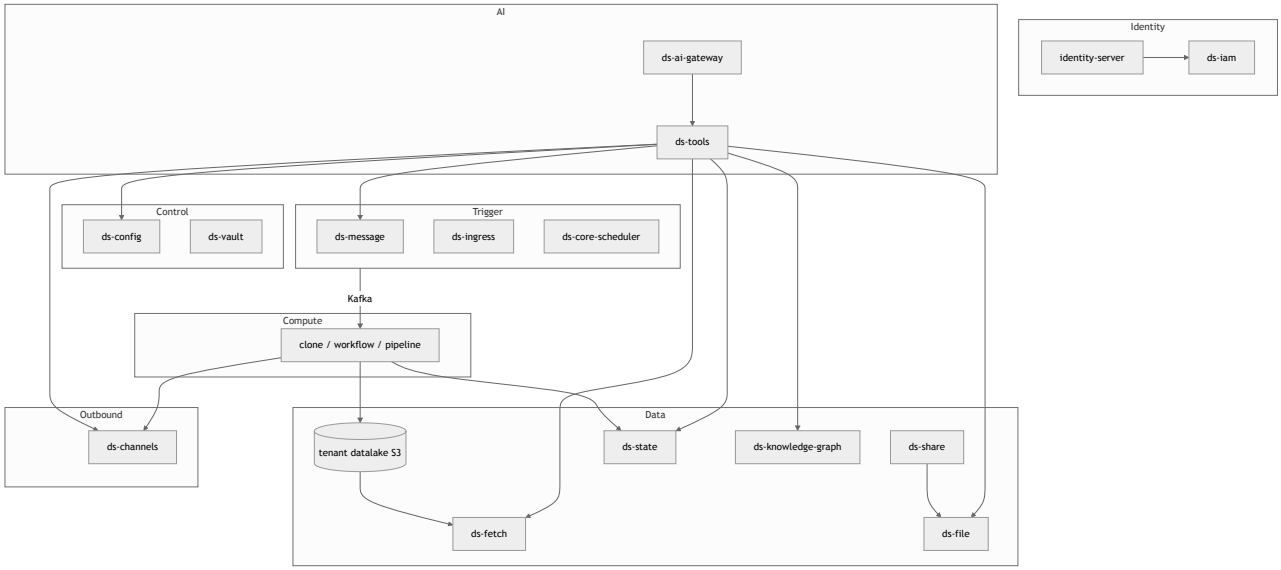
1. Platform overview

AI Commons (also: Commons, AIC, GraspDP, DaaS) is a platform for data, AI and governance. Tenants connect source systems, land data in a per-tenant datalake, run compute on it, and work with it through AI agents that use governed tools.

The platform in numbers

	What
1 IdP	`identity-server` — who you are
14 services	iam, vault, config, message, ingress, fetch, file, share, channels, state, knowledge-graph, tools, ai-gateway, agent
4 compute blocks	`clone`, `workflow`, `pipeline`, `agent` (emerging)
20 tools	`ls cat find info grep schema write rm run status query text_to_sql preview plot inspect extract transcribe frames generate_image websearch`

Mental model



- **Identity** says who you are; **IAM** says what you may do.
- **Config** is the single source of truth: tenants, datasets, compute blocks, schedules, events, linked services, billing and AI policy.
- **Triggers** put a run request on Kafka. **Compute blocks** do the work.
- **Data** lives in the tenant bucket (bronze/silver/gold), in files, in state and in the knowledge graph.
- **Share** creates expiring capability links without weakening source access.
- **Channels** sends email, SMS, Slack and Teams messages to verified channels only (emerging).
- **AI** goes through the gateway, which enforces policy and calls tools.

Environments and URLs

Every API follows one convention:

Env	API	IdP
prod (default)	`https://grasp-daas.com/api/{service}/{version}`	`https://auth.grasp-daas.com`
dev (explicit)	`https://grasp-daas.com/api/{service}-dev/{version}`	`https://auth-dev.grasp-daas.com`

Every service exposes `health-check/`, `version/` and `docs/` (OpenAPI). Example: `https://grasp-daas.com/api/config/v3/docs/`.

Tenants and the public tenant

- A **tenant** is the isolation boundary: its own bucket `ds-<tenant_uuid>-<env>`, its own config rows, secrets, files and graph.
- The **platform tenant** may mark rows `issuer: "public"`. Public rows are readable by every tenant. That is how shared content (playbooks, skills, courses, this book) reaches everyone.
- `public` is not a tenant you log into; it is a visibility flag.

2. Security and logging in

Identity vs authorization

Concern	Owner	Mechanism
Who are you?	`identity-server` (Django + OAuth Toolkit)	RS256 JWT
What may you do?	`ds-iam` + `ds-go-policy` in every service	Policies, evaluated per request

The token carries **identity only**. Permissions are never in the token.

Logging in

1. **SSO (preferred).** Open any Commons app and enter your work email. The app sends you to `{IdP}/oidc/authorize` (Authorization Code + PKCE, S256) with `login_hint=<email>`. Your domain routes you to Entra ID / OIDC / GitHub, and you return via `{IdP}/sso/callback`.
2. **Password + MFA.** `POST {IdP}/auth/login/` → if TOTP is enrolled, `POST {IdP}/mfa/login/` with the ephemeral token.

Check who you are: `GET {IdP}/auth/me/` or `GET {IdP}/oauth/userinfo/`.

Getting a token for scripts and services

Client credentials, one tenant per client. Ask the platform team for an OAuth client and keep the secret in your keychain:

```
IDP=https://auth-dev.grasp-daas.com
JWT=$(printf '%s' "$CLIENT_SECRET" | curl -sS --fail "$IDP/oauth/token/" \
  -d grant_type=client_credentials --data-urlencode "client_id=$CLIENT_ID" \
  --data-urlencode "client_secret@-" | jq -r .access_token)
```

Narrow the audience to one API with RFC 8707: `-d "resource=https://grasp-daas.com/api/config-dev/v3"`.

Grant	Use
`authorization_code` + PKCE	Interactive apps, agents
`client_credentials`	Machines, CI, MCP clients (no refresh token)
`refresh_token`	Interactive only

Scopes: `openid`, `read` (default), `write`. Access token lives **1 h**, refresh **1 d**.

What is in the token

Claim	Meaning
`iss`	IdP host for the env
`sub`	email (user) or `client_id` (app)
`cls`	`user` or `app`
`rsc`	`<tenant_uuid>:<tenant_name>` — your tenant
`aud`	`https://grasp-daas.com` (+ requested resources)

Keys: `{IdP}/oauth/.well-known/jwks.json`. Discovery: `{IdP}/.well-known/openid-configuration`.

How a request is authorized

```
Bearer JWT
 → ds-go-echo-middleware   verify iss, exp, kid; parse cls/sub/rsc     → 401 on failure
 → ds-go-iam-middleware    fetch effective policies from ds-iam (cached)
 → ds-go-policy            Decide / Constrain; default deny, deny wins → 403 on deny
                                                                       → 503 if IAM
unreachable
```

IAM model

AWS-IAM-like:

Primitive	Role
Group	Membership container
Member	IdP `sub` → group
Policy	Allow/deny on actions `{service}:{operationId}` over CRN patterns
Binding	Group ↔ policy

Example action: `config:createWorkflow`, `message:createMessage`, `ai-gateway:callMcpTool`. In patterns, `aic` means "the caller's tenant". New tenants get `IAMFullAccess` at bootstrap. IAM API: `https://grasp-daas.com/api/iam/v1/docs/`.

SCIM. Entra pushes users and groups to `{IdP}/scim/v2/`; groups sync into IAM. SCIM carries membership only; an unbound synced group has **no permissions**.

Secrets

- Never put secrets in config, code or prompts. Store them in **ds-vault**.
- Vault encrypts per secret (KMS-wrapped DEK, AES-256-GCM, AAD bound to tenant + key). Plaintext is never returned by the API; services resolve it server-side.

- AWS rotates KMS key material automatically. Planned Vault lifecycle work adds reviewed KEK replacement and policy-driven DEK re-encryption; these protect stored envelopes but do not replace the external credential.
- Reference a secret from config with `@{get_secret, <key>}`.
- MCP servers and linked services use `credential_ref` → vault key.

Rules

1. Use SSO with MFA. Never share tokens; they are bearer credentials.
2. Use dev for experiments. Prod writes need a reason and a reviewer.
3. Least privilege: request the IAM policy you need, nothing broader.
4. Platform-tenant tokens publish to every tenant. Treat them as root.
5. Report suspected leaks immediately and rotate the secret in vault.

3. The services

The IdP and 14 platform services. HTTP API base URL pattern: `https://grasp-daas.com/api/{service}[-dev]/{version}`. All APIs require `Authorization: Bearer <JWT>` except `health-check/` and `version/`. Each has OpenAPI docs at `<base>/docs/`.

#	Service	Path	Purpose
—	identity-server	`auth[-dev].grasp-daas.com`	Login, tokens, SSO, SCIM
1	ds-iam	`iam/v1`	Groups, members, policies, actions
2	ds-vault	`vault/v1`	Secrets
3	ds-config	`config/v3`	Single source of truth
4	ds-message	`message/v1`	On-demand trigger
5	ds-ingress	`ingress/v1`	Webhook trigger
6	ds-fetch	`fetch/v1`	SQL over the datalake
7	ds-file	`file/v2`	Files and metadata
8	ds-share	`share/v1`	Expiring one-time capability links
9	ds-channels	`channels/v1` (emerging)	Email, SMS, Slack, Teams to verified channels
10	ds-state	`state/v1`	Key/value state, job status
11	ds-knowledge-graph	`knowledge-graph/v1`	Nodes, edges, evidence, retrieval
12	ds-tools	`tools/v1`	The 20 tools (internal)
13	ds-ai-gateway	`ai-gateway/v1`	LLM, MCP, policy, tool loop
14	ds-agent	emerging	Fenced agent compute block

---

identity-server — the IdP

Issues identity JWTs. See chapter 2.

Endpoint	Use
`POST /oauth/token/`	Get a token
`GET /oidc/authorize`	SSO entry
`POST /auth/login/`, `POST /mfa/login/`	Password + MFA
`GET /auth/me/`	Who am I
`/scim/v2/Users`, `/scim/v2/Groups`	Provisioning
`/oauth/.well-known/jwks.json`	Signing keys

1. ds-iam — authorization

Stores who may do what. Evaluation happens inside each service via `ds-go-iam-middleware`.

Endpoint	Use
`/action/`	Global action catalog (`{service}:{operationId}`)
`/group/`, `/member/`	Groups and memberships
`/policy/`, `/map/group-policy/`	Policies and bindings
`/principal/{id}/policies/`	Effective policies for a principal

2. ds-vault — secrets

Envelope-encrypted secrets with per-secret ACL (`owners`, `viewers`). Plaintext is never returned.

Endpoint	Use
`/secret/`	Create / list secrets
`/secret/{id}/`	Read metadata, update, delete
`/store/`	Available backends (`ds_vault`, `aws_ssm`)

Reference from config: `@{get_secret, <key>}`. Go SDK: `ds-vault-go-sdk`; Python: `ds-vault-py-sdk`.

3. ds-config — the source of truth

Everything a tenant *is* and *runs*. Changes are published on Kafka as `resource.changed`.

Area	Endpoints
Tenancy	`/tenant/`, `/owner/`, `/contact/`
Data plane	`/linked-service/`, `/dataset/`, `/schema/`, `/record/`, `/vpn-connection/`
Compute blocks	`/clone/`, `/workflow/`, `/workflow/task/`, `/pipeline/`, `/runtime-profile/`
Triggers	`/schedule/`, `/event/`, `/map/{clone,workflow,pipeline}-{schedule,event}/`, `/map/workflow-task-dependency/`
Queue audit	`/enqueued-job/`
AI policy	`/llm/profile/`, `/llm/model/`, `/llm/provider/`, `/llm/tool/`, `/llm/skill/`, `/llm/mcp-server/`, `/llm/mount/`, `/llm/policy-tier/`, `/llm/rate-limit-tier/`
Billing	`/billing/*`, `/tenant/billing/*`
Learning	`/exam/*`

4. ds-message — on-demand trigger

Publishes a compute-block run request to Kafka with a full config snapshot.

```
POST /message/{topic}/
{"id": "<clone|workflow|pipeline uuid>", "payload": {...}}
```

`GET /topics/` lists topics. IAM: `message:createMessage` + read on the block.

5. ds-ingress — webhook trigger

Receives external webhooks, verifies the signature with the secret from vault, and publishes to Kafka.

Endpoint	Use
`POST /webhook/{event_id}/`	Signed webhook for a Config `event`
`POST /oauth/webhook/{event_id}/`	Webhook authenticated with OAuth

For a provider not listed, ask the platform team.

6. ds-fetch — the datalake gate

Read-only SQL over the tenant's bronze/silver/gold Parquet (DuckDB). `ds-fetch-indexer` keeps the file index fresh.

Endpoint	Use
`POST /query/`	Run SQL `{"sql": "SELECT ... FROM gold.\"<dataset>\" LIMIT 100"}`
`POST /query/validate/`	Validate only
`GET /dataset/`, `/dataset/{name}/`	List / describe datasets

Python: `ds-fetch-py-sdk` (includes the `aic-fetch` CLI).

7. ds-file — files

Metadata in Postgres, bytes in S3. Unique on `(tenant, file_path, version)`.

Endpoint	Use
`POST /file/`	Create metadata (`file_path`, `status` required; `tags`, `metadata`, `issuer`)
`PUT /file/{id}/content/`	Upload bytes (`application/octet-stream` or `text/plain`)
`GET /file/{id}/content/`	Download (supports `Range`)
`GET /file/?file_path_prefix=&tag.k=v&meta.k=v`	List / filter
`POST /search/`	Full-text search

Reserved prefixes: `playbook/`, `courses/`, `book/`.

8. ds-share — one-time sharing

Creates short-lived capability links for DS File artifacts. The creator authenticates normally; the recipient does not need a platform account.

Endpoint	Use
`POST /share/`	Create a grant with TTL and use limit
`GET /share/`, `GET /share/{id}/`	Inspect the creator's grants
`DELETE /share/{id}/`	Revoke a grant
`GET /redeem/{token}`	Atomically redeem and stream the artifact

Only token hashes are stored. DS File remains authoritative for the bytes, and DS Share must not retain creator credentials or expose storage URLs.

9. ds-channels — outbound messaging

Emerging. Sends email, SMS, Slack and Teams messages to **registered, verified channels** only; callers never pass a raw address. Email and SMS verify by double opt-in, Slack and Teams by tenant-admin install. Provider credentials live in ds-vault. Clients are the `channels.send` tool, pipeline `notify` and ds-agent job notifications.

Endpoint	Use
`/channel/`, `/channel/{id}/`	Register, list, update, remove channels
`POST /channel/{id}/verify/`	Re-send the verification challenge
`POST /message/`	Send; `Idempotency-Key` required, returns `202 accepted`
`GET /message/`, `/message/{id}/`	Delivery status

IAM: `channels:sendMessage` on the channel. Lifecycle events: `ds.consumption.channels.message.{sent,delivered,failed}.v1`.

10. ds-state — state and job status

Versioned key/value state and the status of every compute-block run.

Endpoint	Use
`/cb-job/`, `/cb-job/latest/`, `/cb-job/statistics/`	Run status per block
`/cb-stage/`	Pipeline stage status (bronze/silver/gold)
`/state/`	Tenant key/value state
`/search/`	Search state

11. ds-knowledge-graph — knowledge

Typed nodes, directed edges, evidence, and hybrid (keyword + vector) retrieval. Backs `grep` and playbook discovery.

Endpoint	Use
`/node/`, `/node/{id}/`	Nodes (`type`, `name` required)
`/node/{id}/evidence/`	Citations for a node
`/node/{id}/neighbour/`	Graph neighbours
`/edge/`	Edges (`type`, `source_id`, `target_id`)
`POST /graph/retrieve/`	RAG retrieval with hop expansion and citations
`POST /search/`	Simple search

12. ds-tools — the execution plane

Implements the 20 tools (chapter 4). **Not public**: only `ds-ai-gateway` can reach it.

Endpoint	Use
`GET /tools/`	Catalogue
`GET /tools/{tool_id}/`	Schema
`POST /tools/{tool_id}/invoke/`	Invoke (JSON or SSE)

Calls other services with the **caller's** bearer, so IAM applies end to end.

13. ds-ai-gateway — the AI control plane

One entry point for all AI. Loads tenant policy from Config (`/llm/profile/{id}/resolve/`), enforces models, tools, rate limits and approvals, and runs the tool loop.

Endpoint	Use
`POST /openai/v1/chat/completions`	OpenAI-compatible chat
`GET /openai/v1/models`	Models allowed for your tenant
`POST /anthropic/v1/messages`	Anthropic-compatible
`POST /mcp`	MCP server: governed tools for Cursor / Claude (chapter 7)

Gateway chat can retain short facts only when the user explicitly asks through `remember`; `forget` removes one fact. Facts belong to one user in one tenant and differ from `recall`, which searches earlier session summaries. The MCP endpoint does not receive this memory.

---

14. ds-agent — fenced agent compute

Emerging agent compute block built on the gateway harness.

Supporting services

Service	Role
`ds-core-scheduler`	Publishes due `enqueued_job` rows to Kafka
`ds-clone-dispatcher`, `ds-workflow-dispatcher`, `ds-pipeline-router`	Dispatch compute blocks
`ds-workflow-manager`	Runs workflow DAGs
`ds-pipeline-{bronze,silver,gold}-service`	Medallion stages
`ds-data-partition-provision`	Creates bucket, queues, linked services for new tenants
`ds-scim-group-sync`, `idp-kafka-broker`	Identity events → IAM
`ds-skill-sink`, `ds-skill-autocapture`	Capture conversations and mine skills
`ds-core-billing-writer`	Usage events → billing
`ds-agent`	Agent compute on the gateway harness
`ds-client`	AIC front end + BFF (courses, playbooks UI)

4. The tools

20 tools, implemented in `ds-tools`, governed by `ds-ai-gateway`, granted per tenant in Config (`/llm/profile/` → `allowed_tools`). You use them through an AI chat in Commons, or through MCP in Cursor / Claude (chapter 7).

Commons tools are provider-neutral contracts. Approved community providers can implement a backend behind an existing tool without changing how agents call it. New tools and LLM providers enter through reviewed contracts, compatibility and security checks, tenant enablement, IAM, profiles, and Vault-backed credentials—not direct ungoverned execution.

The path model

Most tools operate on one virtual filesystem across services:

Path	Backing service
`/config/...`	ds-config (datasets, workflows, pipelines, schedules ...)
`/files/...`	ds-file
`/state/...`, `/state/jobs/<uuid>`	ds-state
`/kg/...`	ds-knowledge-graph
`/datasets/...`	ds-fetch (lakehouse)
`/mnt/<mount>/...`	External mounts (e.g. GitHub) via `/llm/mount/`

The 20 tools

Group	Tool	What it does
Navigate	`ls`	List children at a path
	`find`	`ls` with filters (`where`, tags, time)
	`cat`	Read one resource
	`info`	Resource + context (neighbours, jobs, previews)
	`grep`	Hybrid keyword + vector search over `/kg`, `/state`, `/files`
Change	`schema`	JSON Schema + example for a path — call before `write`
	`write`	Create or update at a path (approval required on many paths)
	`rm`	Delete at a path (approval required)
Compute	`run`	Fire a clone, workflow or pipeline (via ds-message)
	`status`	Poll a run (`/state/jobs/<uuid>`)
Data	`preview`	Dataset shape + up to 5 sample rows
	`query`	Run SQL on `/datasets/**` (ds-fetch)

Group	Tool	What it does
	`text_to_sql`	Natural language → SQL (does not execute)
	`plot`	SQL or table → chart file in ds-file
Media	`inspect`	Ask a vision model about a file
	`extract`	Text from PDF (`pdftotext`)
	`transcribe`	Speech-to-text from audio/video
	`frames`	Video → still frames
	`generate_image`	Prompt → image in ds-file
Web	`websearch`	Public web search

Patterns that work

Goal	Sequence
Understand something	`ls` → `info` → `cat`
Find something	`grep "<words>"` → `cat`
Create config	`schema` → `write` → `cat` to verify
Run and watch	`run` → `status` until `completed` / `failed`
Answer a data question	`preview` → `text_to_sql` → `query` → `plot`
Read a document	`extract` (PDF) or `inspect` (image)

Gateway meta-tools

When chatting through the gateway harness, the model also sees:

Tool	Purpose
`discover_tools`	Find tools, active skills and playbooks
`expand_tool`	Load a tool schema, skill, or playbook body
`recall`	Recall earlier sessions of the same user
`remember`	Save one short fact when the user explicitly asks
`forget`	Remove one fact when the user explicitly asks
`skill_invoke_<id>`	Run a saved platform skill in one call
`mcp_<slug>__<tool>`	Tool from a tenant-registered external MCP server

`remember` stores at most 30 facts of 300 characters for one user in one tenant. Facts are redacted before storage and supplied to later chat runs as context, not instructions. Plan mode refuses `remember` and `forget`; `remember` is also refused after another tool has returned output in the run. This prevents a web page, file, or tool result from silently planting memory. `forget` removes the fact equal to the text, else the only fact containing it; when several contain it, nothing is removed and the matches come back so the assistant can ask which one.

Facts answer “what did I ask you to always know about me?” `recall` answers “what did I do in earlier sessions?” Fast mode, client-supplied tools, `tool_choice: none`, and the MCP endpoint do not use facts.

Governance

- A tool is callable only if the tenant profile grants it **and** IAM allows the underlying service action with **your** token.
- `write`, `rm` and `run` ask for confirmation on sensitive paths.
- Every call emits a usage event for billing and skill mining.

Specs per tool: `ds-tools/docks/tools/<tool>.md`. Wire contract: `ds-go-tools-client`.

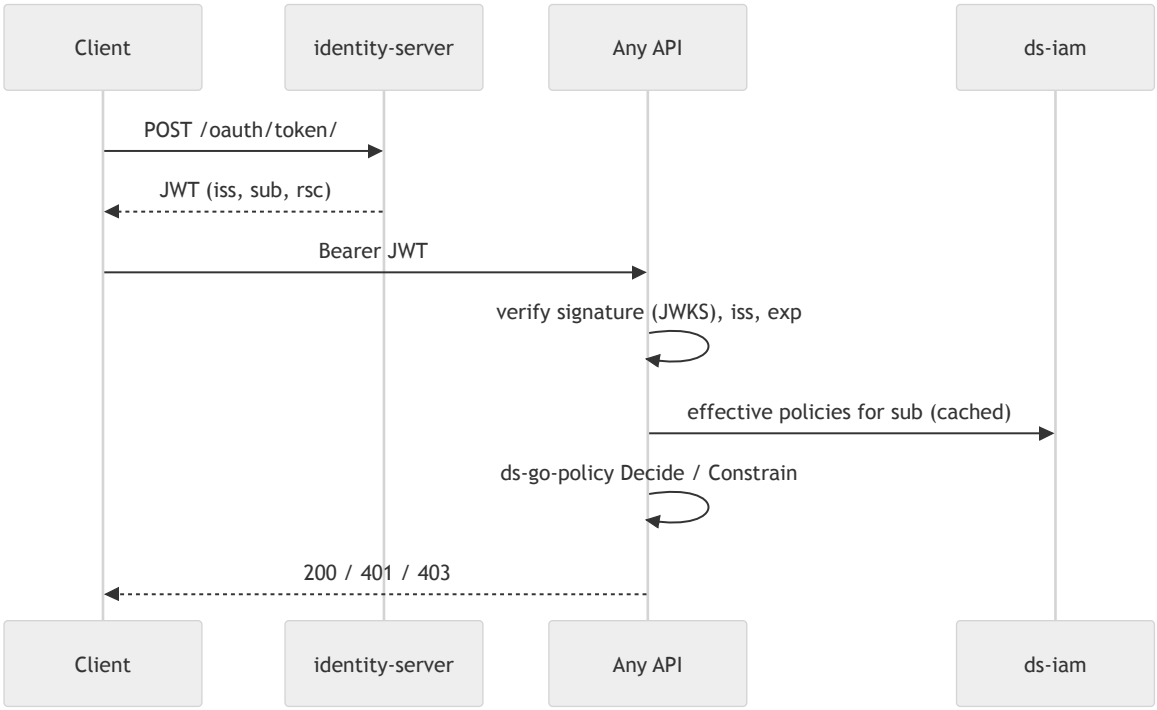
5. How it is wired together

Four flows explain the whole platform: **request**, **compute**, **AI**, **config change**.

Edge

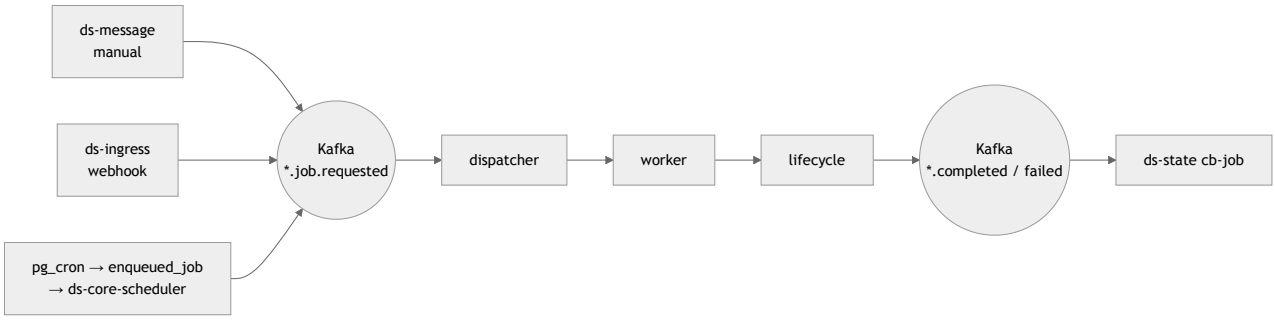
Internet → path-based routes `/api/{service}[-dev]/{version}` → services. `ds-tools` has no public route; only the AI gateway reaches it. Sandboxes and agent workloads run isolated from the APIs. `ds-channels` exposes only its verification links and signed provider callbacks without a JWT.

1. Request flow (every API call)



2. Compute flow (every block run)

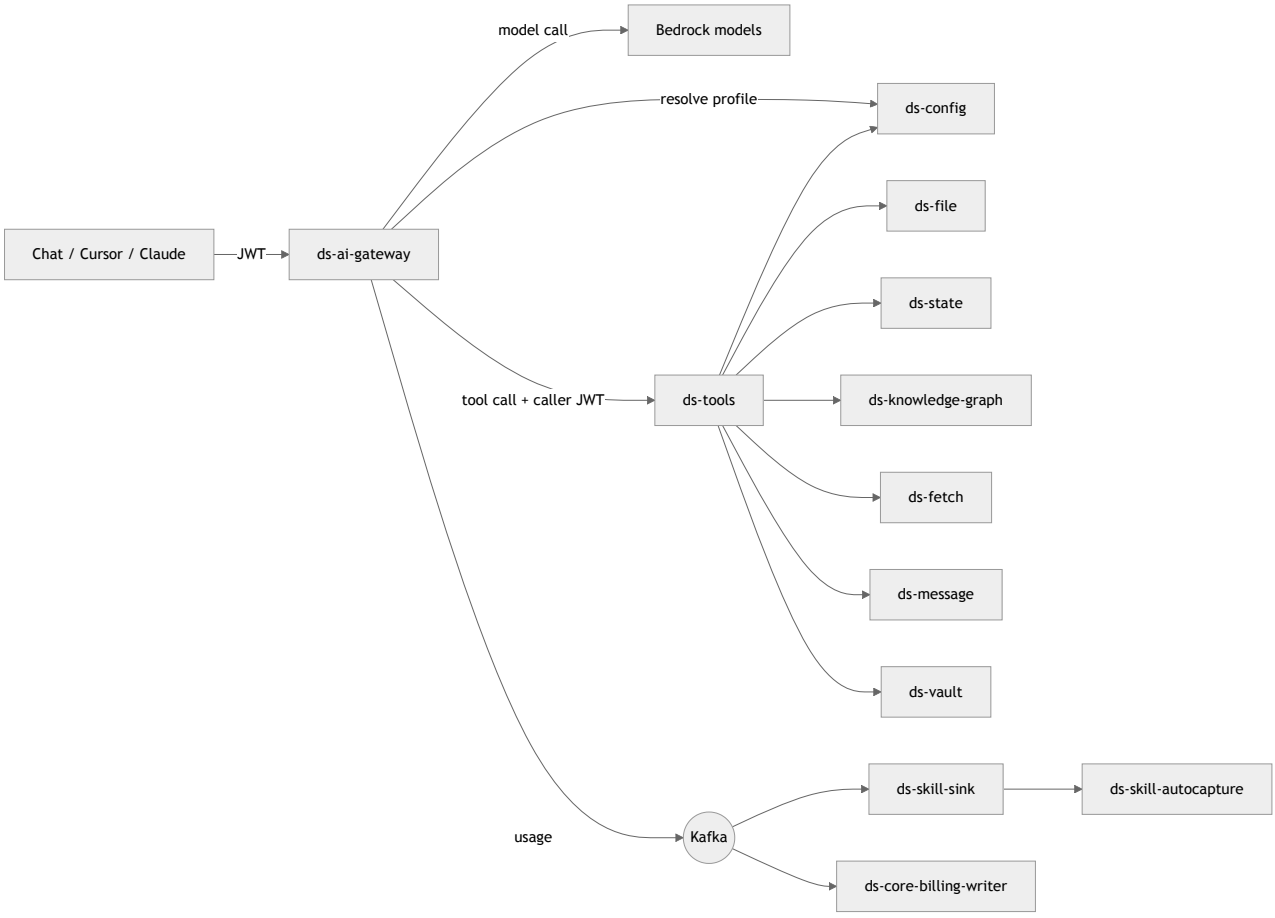
All three blocks follow one shape: **trigger** → **Kafka** → **dispatcher** → **worker** → **lifecycle** → **ds-state**.



Block	Request topic	Dispatcher	Worker
clone	`ds.clone..job.requested.v1`	`ds-clone-dispatcher`	clone service (Fargate/Lambda)
workflow	`ds.workflow.pipeline.job.requested.v2`	`ds-workflow-dispatcher`	`ds-workflow-manager`
pipeline	`ds.pipeline..job.requested.v2`	`ds-pipeline-router`	bronze → silver → gold services

The request event carries a **full snapshot** of the block's config (tasks, datasets, linked services with secret refs, VPN). Workers do not call Config at runtime.

3. AI flow



- The gateway decides **which** models and tools; IAM decides **whether** each underlying call is allowed, using the caller's token.
- Conversations and trajectories are mined into candidate skills.

4. Config change flow

Write to ds-config → row committed → `resource.changed` on Kafka → consumers refresh (gateway policy cache, billing writer, provisioning).

New tenant: identity event → `ds-data-partition-provision` → bucket `ds-<tenant>-<env>`, SQS for the fetch indexer, default linked services, vault secrets, IAM bootstrap.

Kafka conventions

- Envelope and SDK: `ds-event-streams`, `ds-event-stream-go-sdk`.
- Topic shape: `<domain>.<area>.<entity>.<event>.v<N>` (e.g. `idp.identity..group.created.v1`, `ds.pipeline..job.completed.v2`).
- Producers own the schema; consumers must tolerate additive fields.

Shared code

Go services start from `ds-go-echo-template` and share `ds-go-kit`, `ds-go-echo-middleware` (auth), `ds-go-iam-middleware` + `ds-go-policy` (authz), `ds-config-go-sdk`, `ds-vault-go-sdk`. Same conventions everywhere: Echo, `/api/<svc>[-dev]/vN`, embedded OpenAPI, `BUILDING_MODE=dev|prod`.

6. Triggers

A trigger turns "run this block" into a Kafka request event. There are three.

Trigger	Service	Stored as	Typical use
Manual	`ds-message`	nothing (direct publish)	Ad-hoc runs, the `run` tool, UI buttons
Schedule	pg_cron → `enqueued_job` → `ds-core-scheduler`	`schedule` + `map_<block>_schedule`	Nightly loads, recurring checks
Event / webhook	`ds-ingress`	`event` (type `webhook`) + `map_<block>_event`	Source system pushes a change

`enqueued_job.trigger_type` is one of `manual`, `schedule`, `event`.

Manual

```
curl -X POST "$MESSAGE/message/ds.workflow.pipeline.job.requested.v2/" \
  -H "Authorization: Bearer $JWT" -H "Content-Type: application/json" \
  -d '{"id":"<workflow_id>","payload":{"rows":[{"a":1}]}}'
```

ds-message loads the block with IAM-scoped SQL, builds the full snapshot, sets `context.type = "manual"` and publishes. Response contains `job_id` — poll it in ds-state. `payload` reaches the workflow as the `payload` variable.

Schedule

1. Create the schedule (`cron` **or** `rate`, never both):

   ```
   POST /schedule/
   {"name":"nightly-02","status":"active","type":"cron","cron":"0 2 * *
   *","timezone":"Europe/Oslo"}
   ```

2. Map it to a block: `POST /map/pipeline-schedule/ {"pipeline_id":"…","schedule_id":"…"}` (or `/map/workflow-schedule/`, `/map/clone-schedule/`). Pipeline maps may carry `settings` to override the load mode per schedule.

3. Every minute pg_cron calls `dispatch_{clone,workflow,pipeline}_schedules()`. Due, active schedules on active blocks insert one `enqueued_job` (deduplicated on `(cb_id, scheduled_minute)`).

4. `ds-core-scheduler` picks `queued` rows, marks them `running` and publishes to the row's topic. It never marks success; the worker outcome does.

Audit: `GET /enqueued-job/?cb_type=pipeline`. `start_at` / `end_at` travel with the job; consumers enforce them, not the dispatcher.

Event / webhook

1. Store the provider's signing secret in ds-vault.
2. Create the event:

   ```
   POST /event/
   {
     "name": "tripletex-accounts",
     "type": "webhook",
     "status": "active",
     "settings": {
       "type": "webhook",
       "provider": "tripletex",
       "validate_payload": true,
       "vault_ref": "<vault key>",
       "vault_id": "<vault secret id>"
     }
   }
   ```

   Providers with built-in verification: `xledger`, `tripletex`, `hubspot_v1..v3`, `jobbnorge`, `visma_bnxt`, `api_key`.

3. Map it: `POST /map/workflow-event/` (or pipeline / clone).
4. Point the source system at `POST https://grasp-daas.com/api/ingress/v1/webhook/<event_id>/`.
5. ds-ingress verifies the signature and publishes a request for every block mapped to the event.

Pipelines triggered by webhook run in **webhook mode**: bronze is skipped and the pushed payload goes straight to silver → gold.

Following a run

Where	What
`GET /api/state/v1/cb-job/?…` / `cb-job/latest/`	Status per run
`GET /api/state/v1/cb-stage/`	Pipeline stage status
`status` tool	Same, from chat or MCP
Kafka `*.job.completed` / `*.failed`	Outcome events

Rate limits

All three triggers enforce the tenant's compute-block rate limits before a run starts, so the same limits apply whether a run comes from `ds-message`, a schedule or `ds-ingress`.

Tier. Each tenant has one tier, a pool shared by every clone, pipeline, workflow and agent it runs. Without a choice it runs on `starter`. A tenant may pick any tier at or below its service level (`tenant.service_level_id`, `free` by default). A limit of 0 means no limit.

Tier	Runs at once	Starts/min	Starts/day	Queue	Burst	Weight
`starter`	50	100	20,000	1,000	1×	1
`pro`	200	500	200,000	10,000	1.25×	2
`enterprise`	1,000	2,000	none	50,000	1.5×	4
`unlimited`	none	none	none	none	1×	8

Burst lets a quiet tenant briefly exceed starts per minute. Weight is the tenant's share when the platform is busy.

Allocations (optional) share the pool inside a tenant, per block type or per block (the block's own allocation wins):

- `priority`: higher starts first when runs wait; ties go oldest first. Default 0.
- `reserved runs`: slots from "runs at once" that only this block or type may use. The total can't exceed the tier's runs at once.

Priority picks who is next; only reserved runs guarantee a free slot. Example: "pipelines never wait on agents" means pipelines get priority 10 and 2 reserved runs.

Over a limit, the run waits in the tenant's queue. When the queue is full, new runs are rejected or deferred. Tier and allocation changes take up to 5 minutes to apply. Counters are per tenant, so changing tier does not reset them. Tiers, pools and allocations are set by the platform team (`cb_rate_limit_tier`, `cb_rate_limit`, `cb_rate_limit_allocation`). They have no API yet.

Gotchas

- Inactive schedule **or** inactive block = nothing runs.
- A run that hasn't started may be queued by rate limits. Triggering it again adds another run to the queue.
- Workflow topics are `v2`; old docs mentioning `ds.workflow..job.requested.v1` are stale.

7. Setting up MCP

The gateway is an **MCP server**. Connect Cursor or Claude to it and their own model can call your tenant's governed tools (chapter 4). No gateway LLM runs on this path; the gateway only enforces grants, rate limits and audit, and forwards your token to ds-tools.

	prod	**dev**
MCP URL	`https://grasp-daas.com/api/ai-gateway/v1/mcp`	`https://grasp-daas.com/api/ai-gateway-dev/v1/mcp`
Token URL	`https://auth.grasp-daas.com/oauth/token/`	`https://auth-dev.grasp-daas.com/oauth/token/`

Transport: **Streamable HTTP**, stateless `POST /mcp` (`GET` returns 405 by design). Methods: `initialize`, `ping`, `tools/list`, `tools/call`.

1. Get credentials

Ask the platform team for an OAuth client (`client_id`, `client_secret`) on your tenant, and make sure your tenant's LLM profile grants the tools you need (`allowed_tools`). Keep the secret in your keychain or vault, never in a repo.

```
export COMMONS_JWT=$(printf '%s' "$CLIENT_SECRET" | curl -sS --fail https://auth.grasp-daas.com/oauth/token/ \
  -d grant_type=client_credentials --data-urlencode "client_id=$CLIENT_ID" \
  --data-urlencode "client_secret@-" | jq -r .access_token)
```

Tokens last one hour. Re-run before a session, or wrap it in a shell function.

2. Connect

Cursor

`~/.cursor/mcp.json` (global) or `.cursor/mcp.json` (project):

```
{
  "mcpServers": {
    "commons": {
      "url": "https://grasp-daas.com/api/ai-gateway-dev/v1/mcp",
      "headers": { "Authorization": "Bearer ${env:COMMONS_JWT}" }
    }
  }
}
```

Start Cursor from a shell where `COMMONS_JWT` is set, then check **Settings** → **MCP**: `commons` should be green with its tools listed.

Claude Code

Install the `commons` plugin (chapter 8). It connects this server, refreshes tokens from your keychain, and adds the skills and the data-safety guard. In a session, `/mcp` shows the server.

Without the plugin:

```
claude mcp add --transport http commons \
  https://grasp-daas.com/api/ai-gateway-dev/v1/mcp \
  --header "Authorization: Bearer $COMMONS_JWT"
```

Claude Desktop

Desktop needs the `mcp-remote` bridge (`claude_desktop_config.json`):

```
{
  "mcpServers": {
    "commons": {
      "command": "npx",
      "args": ["-y", "mcp-remote",
               "https://grasp-daas.com/api/ai-gateway-dev/v1/mcp",
               "--header", "Authorization: Bearer ${COMMONS_JWT}"],
      "env": { "COMMONS_JWT": "<JWT>" }
    }
  }
}
```

Microsoft Copilot Studio

Copilot exposes the whole MCP server as one `InvokeServer` capability; its orchestrator selects the underlying Commons tool. Prefer the existing, administrator-approved **AI Commons MCP** connector. For a new connector, start in dev and use manual OAuth with a registered `authorization_code` client, `https://auth-dev.grasp-daas.com/oauth/{authorize,token}/`, and resource `https://grasp-daas.com/api/ai-gateway-dev/v1`.

Put deterministic tool order and path limits in the agent instructions. See the `commons-copilot-agent` playbook for DS-Fetch, image inspection and PDF extraction recipes.

3. Verify with curl

```
MCP=https://grasp-daas.com/api/ai-gateway-dev/v1/mcp
H=(-H "Authorization: Bearer $COMMONS_JWT" -H "Content-Type: application/json")

curl -s "$MCP" "${H[@]}" -d '{"jsonrpc":"2.0","id":1,"method":"initialize",
  "params":{"protocolVersion":"2025-06-18","capabilities":{},"clientInfo":
{"name":"curl","version":"1"}}}'
curl -s "$MCP" "${H[@]}" -d '{"jsonrpc":"2.0","id":2,"method":"tools/list"}'
curl -s "$MCP" "${H[@]}" -d '{"jsonrpc":"2.0","id":3,"method":"tools/call",
  "params":{"name":"ls","arguments":{"path":"/config"}}}'
```

Result	Meaning
`401`	Token missing or expired
`-32602 Unknown tool`	Tool not granted to your tenant
`-32000` + `retryAfter`	Rate limit or concurrency cap
`isError: true`	Tool ran but IAM or input rejected it

4. External MCP servers (gateway as client)

Tenants can plug third-party MCP servers into the gateway so they are governed like platform tools:

1. Store the upstream credential in ds-vault.
2. `POST /api/config/v3/llm/mcp-server/` with the upstream HTTPS URL, auth type (`none`, `bearer`, `header`, `basic`, `oauth2_client_credentials`, `oauth2_authorization_code`, `mtls`) and `credential_ref`.
3. Copy the upstream `tools/list` into the row's `tools` map with `effect: allow` (deny by default).
4. Grant IAM `ai-gateway:callMcpTool` on `crn:<tenant>::ai-gateway::mcp-tool:<slug>/<tool>`.

They appear to models as `mcp_<slug>__<tool>`.

8. Skills and playbooks

Three different things share the word "skill". Know which one you mean.

	Platform skill	**Playbook**	**IDE skill (Cursor / Claude)**
What	Saved, parameterised tool trajectory	Markdown guidance for a task	Instructions for your local coding agent
Lives in	ds-config `/llm/skill/`	ds-file `playbook/<domain>/<slug>.md` + KG node	A portable `<name>/SKILL.md` package
Used by	Gateway models, as `skill_invoke_<id>`	Gateway models, via `discover_tools` → `expand_tool`	Cursor / Claude Code
Shared via	`issuer: "public"`	`issuer: "public"` on file and node	The `commons` Claude Code plugin

Platform skills

A skill is a recorded sequence of tool calls with parameters. The model calls it once; the gateway expands the steps server-side.

```
{
  "slug": "createfile",
  "name": "Createfile",
  "descriptions": {"en": "Upload a file: metadata to /files/, then bytes to
/files/{id}/content/."},
  "parameters": [
    {"name": "file_path", "type": "string", "required": true},
    {"name": "encoding",  "type": "string", "required": true}
  ],
  "trajectory": {"steps": [
    {"id": "meta", "tool_id": "write",
     "inputs": {"path": "/files/", "body": {"file_path": "{{file_path}}", "status":
"active"}},
     "bind_output_to": "meta"},
    {"id": "content", "tool_id": "write",
     "inputs": {"path": "/files/{{meta.data.id}}/content/", "body": {"content": "
{{encoding}}"}}}
  ]},
  "required_tool_ids": ["write"],
  "lifecycle": "active"
}
```

Lifecycle: `draft` → `candidate` → `active` → `deprecated` / `rejected`. Only `active` skills are discoverable; promotion needs `config:promoteSkill`.

Where skills come from: `admin` (hand-written), `model_proposed`, and `auto_captured`. Conversations flow through `ds-skill-sink`, and `ds-skill-autocapture` turns repeated read-only trajectories (used by at least two users) into `candidate` skills for review.

Playbooks

A playbook tells a model **how to approach** a task: when to use it, which skills and tools to prefer, and the steps. Format is SKILL.md-compatible:

```
---
name: Build a datalake
description: Land a source system into bronze, transform to silver, publish gold.
when_to_use: User wants recurring, queryable data from an external system.
prefer_skills: []
related_tools: [schema, write, run, status, preview, query]
---
## Steps
...
```

Publishing takes two writes, and the second one is what makes it discoverable:

1. **File** in ds-file at `playbook/<domain>/<slug>.md`, tags `kind=playbook`, `domain=<domain>`.
2. **KG node** named `playbook:<slug>`, `type: unknown`, `status: active`, `uri: dsfile://<file_id>`, `external_id: <file_id>`, `rag_summary: <description>`.

For Commons-wide playbooks, set `issuer: "public"` on both, using a platform-tenant token. The AIC Management UI does both steps for you.

Writing tips: lead with `when_to_use`, set `prefer_skills`, keep the body short and link to the book for depth. Playbooks are ranked by outcome; one that does not help sessions succeed will sink.

The `commons` plugin for Claude Code

The `grasp-labs/commons` repository ships the `commons` Claude Code plugin. Its portable skills are also installed into Cursor and Codex. Public playbooks are authored and published separately for gateway models; they may reference a reusable skill, but they are not generated copies of IDE guidance.

Part	What it does
Skills	`commons` entry point, safety and tool references, service guides, and task guidance including Copilot, Prism, conversational data, datalakes, workflows and publishing
MCP server	Connects the Commons MCP endpoint; `scripts/auth.sh` mints and caches tokens from the keychain

Part	What it does
Guard hook	Asks before `write` / `rm` / `run`, blocks secret writes and credential reads, and limits tenant data in Claude's context per `data_access`

Install (Claude Code, once per machine):

```
claude plugin marketplace add grasp-labs/commons
claude plugin install commons@commons
```

Then store your OAuth client in the keychain and pick the environment and `data_access` in `/plugin` (the `commons` skill walks through it). Requires `jq` and `curl`.

Cursor and Codex read the same skills. Clone `grasp-labs/commons`, then run:

```
python3 install.py cursor
python3 install.py codex
```

Use `--project <path>` to commit skills with a project. Cursor uses `.cursor/skills/`; Codex uses `.codex/skills/`. The Commons MCP server is configured separately as in chapter 7, and the guard hook is Claude Code only.

Read `commons-safety` before pointing an agent at tenant data. The skill tells the agent *what* to do; MCP lets it *do* it; IAM and the guard bound *what it can see*.

9. Building a datalake

Every tenant has a datalake: Parquet on S3 in bucket `ds-<tenant_uuid>-<env>`, organised as a **medallion**. The `pipeline` compute block fills it; `ds-fetch` reads it.

Layer	Holds	Written by	Path
Bronze	Raw rows from the source, per run	`ds-pipeline-bronze-service`	`datalake/bronze/{pipeline_id}/{job_id}/`
Silver	Typed, cleaned rows (your SQL), per run	`ds-pipeline-silver-service`	`datalake/silver/{pipeline_id}/{job_id}/`
Gold	Published table, merged across runs (SCD2 or snapshot)	`ds-pipeline-gold-service`	`datalake/gold/{dataset_id}/` + `_manifest.parquet`

Format is plain Parquet with a manifest and bloom index, not Delta or Iceberg.

In the Commons UI, a **Prism** is a pipeline and a **Facet** is its Gold dataset. The product flow is therefore dataset → Prism → Facet; the API names remain `dataset` and `pipeline`.

What you configure

```
linked_service ──► bronze dataset ─┐
                                   ├─► pipeline (transform = silver SQL) ─► schedule / event / manual
linked_service ──► gold dataset  ──┘
```

0. Tenant prerequisites

Done automatically when a tenant is verified: bucket, file indexing, default `HTTP` and `AIC-AWS` linked services, vault secrets, tenant IAM user. For on-prem sources, add a VPN: `POST /vpn-connection/`.

1. Linked service — how to reach the source

Credentials are vault references, never literals. Typed providers take a vault path (`/ds/vault/<name>/<tenant_id>/<secret_id>/<env>`); generic fields take `@{get_secret, <key>}`.

```
POST /linked-service/
{
  "name": "tripletex-acme",
  "status": "active",
  "type": "ds.resource.linked-service.tripletex",
  "settings": {
    "consumer_token": "/ds/vault/tripletex-consumer/<tenant_id>/<secret_id>/dev",
    "employee_token": "/ds/vault/tripletex-employee/<tenant_id>/<secret_id>/dev"
  }
}
```

Types: `aws`, `aic-aws`, `azure-storage-account`, `graphql`, `http`, `microsoft-sql`, `postgresql`, `sftp`, `soap`, `xledger`, `simployer`, `powerofficego`, `unimicro`, `tripletex`. Settings per type: `api/openapi/components/schemas/linked_service/settings/`. Examples: `docs/examples/linked-service.json`.

2. Bronze dataset — what to read

```
POST /dataset/
{
  "name": "tripletex-employee",
  "status": "active",
  "type": "ds.resource.dataset.tripletex",
  "linked_service_id": "<tripletex linked service>",
  "settings": { "product_name": "employee" }
}
```

Source types: `tripletex`, `xledger`, `unimicro`, `powerofficego`, `simployer`, `http`, `graphql`, `soap`, `sftp`, `postgresql`, `microsoft-sql`, `azure-blob`, `azure-table`, `aic-file`, `grasp-file` (all `ds.resource.dataset.<type>`). Provider logic lives in `ds-provider-*-py-lib`.

3. Gold dataset — what to publish

```
POST /dataset/
{
  "name": "employee",
  "status": "active",
  "type": "ds.resource.dataset.aic-gold",
  "linked_service_id": "<AIC-AWS linked service>",
  "settings": {
    "primary_key_fields": ["employee_id"],
    "ordering_field": "version",
    "history_mode": "scd2",
    "partition_keys": []
  }
}
```

Setting	Meaning
`primary_key_fields`	Row identity. Cannot change once data exists.
`ordering_field`	Highest value wins; older arrivals are dropped as late.
`history_mode`	`scd2` keeps history (`_valid_from`, `_valid_to`); `none` overwrites. Cannot change once data exists.
`partition_keys`	Hive partitions. Choose early; changes are validated against existing data.
`delete_field`	Optional soft-delete flag column.

4. Pipeline — tie it together

```
POST /pipeline/
{
  "name": "tripletex-employee",
  "status": "active",
  "bronze_id": "<bronze dataset>",
  "gold_id": "<gold dataset>",
  "transform": {
    "sql_query": "SELECT CAST(id AS BIGINT) AS employee_id, firstName AS first_name,
lastName AS last_name, CAST(version AS BIGINT) AS version FROM bronze",
    "non_null_columns": ["employee_id"]
  },
  "snapshot_profile_id": "<runtime profile>",
  "incremental_profile_id": "<runtime profile>",
  "settings": {
    "strategy_policy": { "defaults": { "load_mode": "incremental" } },
    "artifact_policy": { "bronze_retention_days": "14d", "silver_retention_days": "90d"
}
  }
}
```

- **Silver SQL** is a restricted DuckDB `SELECT … FROM bronze [WHERE …]`: one row in, at most one row out. No `*`, joins, aggregates or subqueries. Full grammar: `silver-sql.md`.
- **Runtime profiles** (`/runtime-profile/`) choose executor, memory and network. `cpu` must match `memory_mb`: Lambda goes up to 10240 MB, Fargate from 2048 to 61440 MB.
- **Load mode**: `snapshot` (full read) or `incremental` (checkpointed).

5. Trigger it

Manual, schedule or webhook (chapter 6). Manual:

```
curl -X POST "$MESSAGE/message/ds.pipeline..job.requested.v2/" \
  -H "Authorization: Bearer $JWT" -H "Content-Type: application/json" \
  -d '{"id":"<pipeline_id>"}'
```

`ds-pipeline-router` runs bronze → silver → gold (webhook mode: silver → gold). Watch stages in ds-state `/cb-stage/`.

6. Query it

```
curl -s "$FETCH/dataset/" -H "Authorization: Bearer $JWT"     # find table names
curl -s -X POST "$FETCH/query/" -H "Authorization: Bearer $JWT" \
  -H "Content-Type: application/json" \
  -d '{"sql":"SELECT * FROM gold.\"<table>\" WHERE _valid_to IS NULL LIMIT 10"}'
```

Or from chat / MCP:

1. `preview {rows: 0}` grounds the question in the real schema and returns the table's `sql_ref`.
2. `text_to_sql` drafts one read-only query; it does not execute.
3. Review tables, columns, grain, filters, units and sensitive fields.
4. `query` executes against the same dataset paths; `plot` is optional.

The answer should name its dataset, filters and time range. Aggregate first and ask before returning identifiable rows. See `commons-talk-to-data`.

Design checklist

- [ ] Primary key is truly unique and stable in the source.
- [ ] `ordering_field` increases on every change.
- [ ] `non_null_columns` covers the primary key.
- [ ] `history_mode` chosen deliberately (`scd2` for audit, `none` for snapshots).
- [ ] Partitions only on low-cardinality, query-relevant columns.
- [ ] All credentials are `@{get_secret, …}`.
- [ ] Dev run succeeded and gold row count is plausible before prod.

10. Building a workflow

A **workflow** is a DAG of tasks: read, transform, write, branch, notify, run Python. Use it for operational jobs (sync A → B, enrich, validate, notify). Use a **pipeline** (chapter 9) when the goal is a governed bronze/silver/gold table.

Anatomy

Part	Stored in Config	Purpose
Workflow	`/workflow/`	Name + settings (executor, network, retry, timeout, variables, notify)
Tasks	`/workflow/task/`	One step each: `type`, `settings`, `bindings`, `condition`, `severity`
Dependencies	`/map/workflow-task-dependency/`	Edge: `workflow_task_id` runs after `dependency_id`
Triggers	`/map/workflow-schedule/`, `/map/workflow-event/`	When it runs

At run time ds-message (or the scheduler) compiles these rows into one JSON graph and `ds-workflow-manager` executes it.

Task types

Category	Types (`ds.workflow.task.*`)
IO	`io.read.dataset`, `io.create.dataset`, `io.update.dataset`, `io.upsert.dataset`, `io.delete.dataset`, `io.rename.dataset`, `io.purge.dataset`, `io.list.dataset`
Transform	`operational.transformation.{sql-query, query, filter-columns, filter-values, rename, dtypes, derived-columns, assign, map, join, concat, aggregate, series, extract-xml-value}`
Execution	`operational.execution.python`
Analytics	`operational.analytics.collect-errors`
Input	`functional.input.source` (turns the trigger payload into a table)
Notify	`functional.notification.email`
Control	`control.condition.choice`, `control.condition.channel-guard`, `control.wait.sleep`

Ports per task: call the `schema` tool.

Condition (when a task runs given upstream results): `always`, `all-done`, `all-success`, `any-success`, `all-failure`, `any-failure`. **Severity**: `critical` (failure fails the run) or `minor`.

Bindings wire data into ports:

```
{"source": {"kind": "task-ref", "channel": "success", "ref": {"id": "<task id>"}},
 "target": {"kind": "port", "port": "inputs", "entry": "input_0"}}
```

Sources: `variable` (e.g. `payload`), `task-ref` (output of another task).

Build one: payload → SQL filter → write

Goal: accept rows in the trigger payload, keep Norwegian ones, upsert into a dataset.

```
CFG=https://grasp-daas.com/api/config-dev/v3
H=(-H "Authorization: Bearer $JWT" -H "Content-Type: application/json")
```

1. Workflow

```
curl -s -X POST "$CFG/workflow/" "${H[@]}" -d '{
  "name": "orders-no", "status": "active",
  "settings": {
    "compute_engine": "pandas", "executor": "lambda",
    "network": {"mode": "public"},
    "retry": {"max_attempts": 2, "backoff_seconds": 30},
    "notify": {"on_failure": true, "email": ["you@company.com"]},
    "variables": {}
  }}'
```

2. Tasks

```
# input
curl -s -X POST "$CFG/workflow/task/" "${H[@]}" -d '{
  "workflow_id": "<wf>", "name": "INPUT", "status": "active",
  "type": "ds.workflow.task.functional.input.source",
  "condition": "always", "severity": "critical",
  "bindings": [{"source": {"kind": "variable", "name": "payload"},
                "target": {"kind": "port", "port": "payload"}}],
  "settings": {"format": "semi-structured-json", "kwargs": {"record_path": "rows"}}}'

# filter
curl -s -X POST "$CFG/workflow/task/" "${H[@]}" -d '{
  "workflow_id": "<wf>", "name": "FILTER_NO", "status": "active",
  "type": "ds.workflow.task.operational.transformation.sql-query",
  "condition": "all-success", "severity": "critical",
  "bindings": [{"source": {"kind": "task-ref", "channel": "success", "ref": {"id": "
<input task>"}},
                "target": {"kind": "port", "port": "inputs", "entry": "input_0"}}],
  "settings": {"query": "SELECT * FROM input_0 WHERE country = '\''NO'\''"}}'

# write
curl -s -X POST "$CFG/workflow/task/" "${H[@]}" -d '{
  "workflow_id": "<wf>", "name": "UPSERT", "status": "active",
  "type": "ds.workflow.task.io.upsert.dataset", "dataset_id": "<target dataset>",
  "condition": "all-success", "severity": "critical",
  "bindings": [{"source": {"kind": "task-ref", "channel": "success", "ref": {"id": "
<filter task>"}},
                "target": {"kind": "port", "port": "input"}}],
  "settings": {}}'
```

3. Dependencies

```
curl -s -X POST "$CFG/map/workflow-task-dependency/" "${H[@]}" \
  -d '{"workflow_task_id": "<filter task>", "dependency_id": "<input task>"}'
curl -s -X POST "$CFG/map/workflow-task-dependency/" "${H[@]}" \
  -d '{"workflow_task_id": "<upsert task>", "dependency_id": "<filter task>"}'
```

4. Run

```
curl -s -X POST "https://grasp-daas.com/api/message-
dev/v1/message/ds.workflow.pipeline.job.requested.v2/" "${H[@]}" \
  -d '{"id": "<wf>", "payload": {"rows": [{"id": 1, "country": "NO"}, {"id": 2,
"country": "SE"}]}}'
```

5. Watch: ds-state `/cb-job/latest/`, or the `status` tool.

Add a schedule or webhook later (chapter 6). The workflow does not change.

Run locally (no cloud)

```
cd ds-workflow-manager
uv sync --all-extras --dev
uv run python -m examples.01_run_pipeline     # runs examples/data/workflow.json
```

Edit `examples/data/workflow.json` to prototype a graph before writing Config rows.

Settings reference

Setting	Values
`executor`	`lambda` (short, ≤15 min) or `fargate` (long, heavy)
`network.mode`	`public`, `nat`, `static_ip` (allow-listed egress), `vpn` (on-prem)
`max_concurrent_jobs`	1–100 (default 1)
`retry`	`max_attempts`, `backoff_seconds`
`timeout`	seconds
`variables`	Values available to tasks (e.g. `orgnr`)
`notify`	`on_success`, `on_failure`, `email[]`

Tips

- Prototype locally, then create rows in dev, then promote.
- Name tasks `CATEGORY::ACTION::TARGET`; it makes runs readable.
- Use `minor` severity for optional side effects (notifications).
- Keep secrets in linked services; tasks reference datasets, not credentials.

11. Get started examples

Do these in **dev**, in order. Each takes 5–15 minutes.

```
export IDP=https://auth-dev.grasp-daas.com
export API=https://grasp-daas.com/api
export JWT=$(curl -s -X POST "$IDP/oauth/token/" \
  -d grant_type=client_credentials -d client_id="$CLIENT_ID" -d
client_secret="$CLIENT_SECRET" \
  | jq -r .access_token)
H=(-H "Authorization: Bearer $JWT" -H "Content-Type: application/json")
```

1. Say hello to the platform

```
curl -s "$IDP/auth/me/" -H "Authorization: Bearer $JWT"   # who am I
curl -s "$API/config-dev/v3/tenant/" "${H[@]}"            # my tenant
curl -s "$API/config-dev/v3/version/"                      # no auth needed
```

Decode your token: `jq -R 'split(".")[1] | gsub("-";"+") | gsub("_";"/") | @base64d | fromjson' <<<"$JWT"`. Find `rsc` (your tenant) and `cls`.

2. Connect Cursor to Commons

Follow chapter 7. Then ask Cursor:

> List what is under /config, then show me the most recent pipeline run.

You should see `ls` and `status` calls in the chat.

3. Store and read a file

```
ID=$(curl -s -X POST "$API/file-dev/v2/file/" "${H[@]}" \
  -d '{"file_path":"sandbox/hello.md","status":"active","tags":{"kind":"note"}}' | jq -r
.id)
curl -s -X PUT "$API/file-dev/v2/file/$ID/content/" \
  -H "Authorization: Bearer $JWT" -H "Content-Type: text/plain" --data-binary "# Hello
Commons"
curl -s "$API/file-dev/v2/file/$ID/content/" -H "Authorization: Bearer $JWT"
```

4. Add knowledge to the graph

```
curl -s -X POST "$API/knowledge-graph-dev/v1/node/" "${H[@]}" \
  -d '{"type":"person","name":"<Your Name>","status":"active","props":{"team":"
<team>"}}'
curl -s -X POST "$API/knowledge-graph-dev/v1/graph/retrieve/" "${H[@]}" \
  -d '{"query":"<Your Name>"}'
```

5. Run a workflow with a payload

Build the three-task workflow in chapter 10, then fire it:

```
curl -s -X POST "$API/message-dev/v1/message/ds.workflow.pipeline.job.requested.v2/"
"${H[@]}" \
  -d '{"id":"<workflow_id>","payload":{"rows":[{"id":1,"country":"NO"}]}}'
curl -s "$API/state-dev/v1/cb-job/latest/" "${H[@]}"
```

6. Land data in gold and query it

Follow chapter 9 with an `http` source (e.g. a public JSON API, see `docs/examples/http.json`), then:

```
curl -s -X POST "$API/fetch-dev/v1/query/" "${H[@]}" \
  -d '{"sql":"SELECT count(*) FROM gold.\"<table>\""}'
```

7. Schedule it

```
SID=$(curl -s -X POST "$API/config-dev/v3/schedule/" "${H[@]}" \
  -d '{"name":"hourly","status":"active","type":"cron","cron":"0 * * *
*","timezone":"Europe/Oslo"}' | jq -r .id)
curl -s -X POST "$API/config-dev/v3/map/pipeline-schedule/" "${H[@]}" \
  -d "{\"pipeline_id\":\"<pipeline_id>\",\"schedule_id\":\"$SID\"}"
```

Next hour: `GET /enqueued-job/?cb_type=pipeline` shows the queued run.

8. Ask the AI about your data

```
curl -s "$API/ai-gateway-dev/v1/openai/v1/models" "${H[@]}"   # pick an allowed model
curl -s "$API/ai-gateway-dev/v1/openai/v1/chat/completions" "${H[@]}" -d '{
  "model":"<model>",
  "messages":[{"role":"user","content":"Preview my gold datasets and plot row counts per
dataset."}]}'
```

The gateway discovers tools, calls `preview`, `query` and `plot`, and returns a link to the chart in ds-file.

9. Publish a playbook

Write what you learned as a playbook (chapter 8) and publish it to your tenant first. Once it has been reviewed, the platform team promotes it to Commons.

Done?

You have touched identity, config, files, graph, triggers, both compute paths, the datalake, AI and MCP. Next: take the course *Build with Commons* and read each service's OpenAPI docs at `<base>/docs/`.

12. Reference

URLs

	dev	prod
IdP	`https://auth-dev.grasp-daas.com`	`https://auth.grasp-daas.com`
APIs	`https://grasp-daas.com/api/{svc}-dev/{v}`	`https://grasp-daas.com/api/{svc}/{v}`
MCP	`…/ai-gateway-dev/v1/mcp`	`…/ai-gateway/v1/mcp`
AIC app	`https://dev.aic-project.com`	`https://aic-project.com`

Versions: `config/v3`, `file/v2`, all others `v1`.

Kafka topics

Topic	Meaning
`ds.clone..job.requested.v1`	Run a clone
`ds.workflow.pipeline.job.requested.v2`	Run a workflow
`ds.pipeline..job.requested.v2`	Run a pipeline
`ds.<block>…job.{completed,failed}.v<N>`	Run outcome
`resource.changed`	Config row changed
`idp.identity..group.{created,updated,deleted,members.updated}.v1`	SCIM group events
`ds.consumption.ai-gateway.message.created.v1`	AI conversation messages

Status codes

Code	Meaning
401	Token missing, expired, wrong issuer
403	IAM denied
409	Conflict (e.g. gold schema drift)
422	Validation failed
503	IAM or a dependency unreachable — not a deny

Glossary

Term	Meaning
Commons / AIC	The platform and its shared public content
Tenant	Isolation boundary; `rsc` claim

Term	Meaning
Platform tenant	The tenant that may publish `issuer: "public"`
Compute block	`clone`, `workflow` or `pipeline`
Linked service	Connection + credential refs to a system
Dataset	A readable/writable thing behind a linked service
Medallion	Bronze (raw) → silver (clean) → gold (published)
SCD2	Gold history mode with `_valid_from` / `_valid_to`
cb-job	Run record in ds-state
Profile	Tenant AI policy: models, tools, limits
Platform skill	Saved tool trajectory, `skill_invoke_<id>`
Playbook	Markdown guidance + KG node `playbook:<slug>`
CRN	Resource name used in IAM policies

Where to read more

Topic	Where
API reference	`https://grasp-daas.com/api/{service}[-dev]/{version}/docs/`
OpenID discovery	`{IdP}/.well-known/openid-configuration`
Agent skills	`skills/` in `grasp-labs/commons`
Courses	*Welcome to Commons*, *Build with Commons*